Лабораторийн ажил №4

Урсгал (Threads) ба зэрэгцээ байдал (Concurrency) — Бүлэг 4, Practice Exercises

F.CSM302 Үйлдлийн систем
Гүйцэтгэсэн: Б.Билгүүн (B190910014)

Сурах бичиг: A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018

Дасгал 4.1

Асуулт. Мулти-урсгал (multithreading) ашиглах нь нэг урсгалт шийдлээс илүү гүйцэтгэл үзүүлэх программчлалын гурван жишээ дурд.

Хариулт.

1. **Вэб сервер.** Олон клиентийн хүсэлтийг зэрэг хүлээн авч, хүсэлт бүрийг тусдаа урсгалаар боловсруулна. Нэг урсгалт сервер нэг клиентэд үйлчилж байх хооронд бусад нь дараалалд хүлээнэ. Мулти-урсгалт серверт нэг урсгал дискнээс файл уншиж блоклогдсон байхад бусад урсгал хүсэлтээ үргэлжлүүлэн боловсруулна.
2. **Интерактив GUI программ** (жишээ нь текст засварлагч). Нэг урсгал хэрэглэгчийн оролтыг хүлээн авч, нөгөө нь дэлгэц шинэчилж, гурав дахь нь ар талд үг үсгийн алдаа шалгах, автоматаар хадгалах ажлыг хийнэ. Ингэснээр урт ажил хийгдэж байх үед ч программ хариу үйлдэлтэй (responsiveness) хэвээр байна.
3. **Мулти-цөмт систем дээрх тооцооллын ажил.** Матриц үржүүлэх, зураг/видео боловсруулах, том массив эрэмбэлэх, Monte Carlo симуляц зэрэг ажлыг хэсэг болгон хувааж, урсгал бүрийг өөр цөм дээр зэрэг ажиллуулна. Нэг урсгалт программ зөвхөн нэг цөм ашигладаг бол мулти-урсгалт программ бүх цөмийг ашиглана (scalability).

Дасгал 4.2

Асуулт. Amdahl-ын хуулийг ашиглан 60 хувь нь параллел хэсэгтэй программын хурдсалтыг (a) хоёр цөмтэй, (b) дөрвөн цөмтэй үед тооцоол.

Хариулт.

Amdahl-ын хууль:

$$\text{speedup} \leq \frac{1}{S + \dfrac{1-S}{N}},$$

энд S нь дараалсан (сериал) хэсгийн хувь, N нь цөмийн тоо. Параллел хэсэг 60% тул $S = 0.4$, $1 - S = 0.6$.

(a) $N = 2$: $\text{speedup} = \dfrac{1}{0.4 + \frac{0.6}{2}} = \dfrac{1}{0.4 + 0.3} = \dfrac{1}{0.7} \approx \mathbf{1.43}$ дахин.

(b) $N = 4$: $\text{speedup} = \dfrac{1}{0.4 + \frac{0.6}{4}} = \dfrac{1}{0.4 + 0.15} = \dfrac{1}{0.55} \approx \mathbf{1.82}$ дахин.

Цөмийн тоог хоёр дахин нэмэхэд хурдсалт 1.43-аас 1.82 болж, ердөө 27%-иар өссөн. Цөмийн тоо $N \to \infty$ болоход хурдсалт $1/S = 2.5$ дахинд дөхдөг. Иймд программын дараалсан хэсэг хурдсалтын дээд хязгаарыг тодорхойлно.

Дасгал 4.3

Асуулт. 4.1-р хэсэгт тайлбарласан мулти-урсгалт вэб сервер нь даалгаврын параллелчлал (task parallelism) уу, өгөгдлийн параллелчлал (data parallelism) уу?

Хариулт.

Өгөгдлийн параллелчлал (data parallelism).

Мулти-урсгалт серверт хүсэлт бүрийг боловсруулах шинэ урсгал үүсгэдэг. Урсгал бүр ижил үйлдэл гүйцэтгэдэг: хүсэлт хүлээн авч, хариу буцаана. Харин тэд өөр өөр өгөгдөл буюу өөр клиентийн хүсэлт дээр ажилладаг. Өгөгдлийг хуваан олон цөмд хуваарилж, тус бүр дээр нь ижил үйлдэл гүйцэтгэхийг өгөгдлийн параллелчлал гэдэг.

Харин даалгаврын параллелчлалд урсгал бүр өөр өөр үйлдэл гүйцэтгэнэ. Жишээ нь 4.22-ын программд нэг урсгал дундаж, нөгөө нь хамгийн их, гурав дахь нь хамгийн бага утгыг тооцдог.

Дасгал 4.4

Асуулт. Хэрэглэгчийн түвшний урсгал (user-level thread) ба кернелийн урсгал (kernel-level thread)-ын хоёр ялгааг дурд. Ямар нөхцөлд аль нь илүү тохиромжтой вэ?

Хариулт.

Ялгаа:

1. **Кернел мэдэх эсэх, хуваарилалт.** Хэрэглэгчийн түвшний урсгалын тухай кернел мэддэггүй. Тэдгээрийг хэрэглэгчийн түвшний урсгалын сан (Pthreads, Java threads г.м.) үүсгэж, хуваарилдаг. Кернелийн урсгалыг кернел шууд дэмжиж, CPU хуваарилагч нь хуваарилдаг.

2. **Үүсгэх, солих зардал.** Хэрэглэгчийн түвшний урсгалыг үүсгэх, солиход систем дуудалт шаардлагагүй тул маш хурдан. Кернелийн урсгалын үйлдэл бүр кернел рүү шилжих (систем дуудалт) шаарддаг тул удаан.

3. **Блоклох ба мулти-цөм.** Many-to-one загварт нэг урсгал блоклох систем дуудалт хийвэл бүх процесс блоклогдоно. Мөн мулти-цөмт систем дээр параллел ажиллахгүй. Кернелийн урсгалын нэг нь блоклогдоход бусад нь үргэлжлэн ажиллаж, өөр өөр цөм дээр зэрэг ажиллана.

Аль нь тохиромжтой вэ:

- Хэрэглэгчийн түвшний урсгал нь маш олон жижиг урсгал үүсгэж байнга сольдог, I/O-д бага блоклогддог ажилд тохиромжтой. Жишээ нь нэг цөм дээрх симуляц.
- Кернелийн урсгал нь мулти-цөмт систем дээрх параллел тооцоолол болон блоклох I/O ихтэй программд тохиромжтой. Жишээ нь вэб сервер, өгөгдлийн сан.

Дасгал 4.5

Асуулт. Кернелийн урсгалуудын хооронд контекст сэлгэлт (context switch) хийхэд кернелийн гүйцэтгэх үйлдлүүдийг тайлбарла.

Хариулт.

1. Timer тасалдал эсвэл систем дуудалтаар кернелийн горимд шилжинэ.
2. Одоо ажиллаж байгаа урсгалын контекстыг тухайн урсгалын өгөгдлийн бүтцэд (Linux-д `task_struct`, Windows-д KTHREAD) болон кернелийн стэйкт хадгална. Үүнд CPU регистрүүд (программын тоолуур, стэйкийн заагч, ерөнхий регистрүүд) болон төлөвийн регистр орно.
3. Тэр урсгалын төлөвийг (бэлэн, хүлээлт г.м.) шинэчилж, тохирох дараалалд оруулна.
4. CPU хуваарилагч дараагийн ажиллах урсгалыг сонгоно.
5. Хэрэв шинэ урсгал өөр процессынх бол санах ойн орчныг (хуудасны хүснэгтийн заагч, TLB) солино. Ижил процессын урсгал бол хаягийн орон нь ижил тул энэ алхмыг алгасна. Иймээс урсгал хооронд шилжилт процесс хооронд контекст сэлгэлтээс бага зардалтай.
6. Шинэ урсгалын хадгалсан регистрүүдийг сэргээж, хэрэглэгчийн горимд буцаана. Урсгал зогссон цэгээсээ үргэлжилнэ.

Дасгал 4.6

Асуулт. Урсгал үүсгэхэд ямар нөөц ашиглагддаг вэ? Энэ нь процесс үүсгэхэд ашиглагдах нөөцөөс юугаараа ялгаатай вэ?

Хариулт.

Урсгал үүсгэхэд зөвхөн тухайн урсгалд хамаарах жижиг өгөгдлийн бүтэц хэрэгтэй:

- урсгалын ID болон урсгалын удирдлагын өгөгдлийн бүтэц (төлөв, эрэмбэ);
- регистрийн багц, программын тоолуур хадгалах зай;
- өөрийн стэйк (хэрэглэгчийн горимын болон кернел горимын стэйк);
- урсгалын хувийн санах ой (thread-local storage).

Код, глобал өгөгдөл, heap, нээсэн файлууд, сигналын тохиргоо зэргийг процессынхоо бусад урсгалтай дундаа хэрэглэнэ (resource sharing).

Процесс үүсгэхэд ПУБ (PCB)-ээс гадна бүхэл бүтэн шинэ хаягийн орон хэрэгтэй. Үүнд код, өгөгдөл, heap, стэйкийн хэсгүүд, хуудасны хүснэгт, нээсэн файлын хүснэгт, эрх, нөөцийн бүртгэл орно. Иймд процесс үүсгэх нь ”хүнд”, санах ой их шаарддаг. Урсгал үүсгэх нь ”хөнгөн”, хэдэн арав дахин хурдан бөгөөд бага нөөц зарцуулна (economy).

Дасгал 4.7

Асуулт. Үйлдлийн систем хэрэглэгчийн түвшний урсгалуудыг кернелд many-to-many загвараар, LWP (lightweight process)-ээр дамжуулан харгалзуулдаг гэж үзье. Мөн систем нь real-time системд зориулсан real-time урсгал үүсгэх боломжтой. Real-time урсгалыг LWP-д холбох (bind) шаардлагатай юу? Тайлбарла.

Хариулт.

Тийм, холбох шаардлагатай.

Many-to-many загварт хэрэглэгчийн түвшний урсгал ажиллахын тулд урсгалын сан түүнийг сул LWP дээр хуваарилах ёстой. LWP нь виртуал процессор бөгөөд бүр нэг кернелийн урсгалд хавсаргагдсан байдаг. CPU дээр зөвхөн кернелийн урсгалыг кернел хуваарилдаг.

Хэрэв real-time урсгал LWP-д байнга холбогдоогүй бол ажиллах шаардлагатай болоход бүх LWP бусад урсгалаар завгүй байж болно. Тэгвэл real-time урсгал сул LWP гарч иртэл урьдчилан тооцох боломжгүй хугацаагаар хүлээх болно. Real-time систем хугацааны хатуу хязгаар (deadline) мөрдөх ёстой тул ийм хоцролтыг хүлээн зөвшөөрөхгүй.

Real-time урсгалыг LWP-д холбосноор (bound thread) тэр LWP зөвхөн энэ урсгалд зориулагдана. Ингэснээр урсгал шаардлагатай үедээ кернелийн хуваарилагчаар шууд, хамгийн бага хоцролтоор хуваарилагдана.